

Mini Project - Introduction and VGA Interface

Maryam Hemmati

Department of Electrical, Computer, and Software Engineering
University of Auckland

email: m.hemmati@auckland.ac.nz

COMPSYS 305-Digital Systems Design

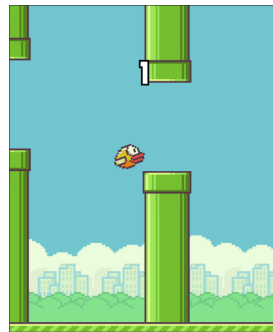
30 March 2026

- ① Mini Project Objective
- ② DE0-CV Board
- ③ VGA Interface
- ④ Graphics Display on VGA Screen

Mini Project Objective

The goal of the mini project is to design a simple game console.

- The game is Flappy Bird.
- The game is implemented on DE0-CV board.
- The game is controlled and played using
 - ▶ A PS/2 mouse
 - ▶ DIP switches on the DE0-CV board
 - ▶ Push-buttons on the DE0-CV board
- The game is displayed on a VGA screen.
 - ▶ with a resolution of **640x480** pixels



Mini Project Objective

Game Description

- The bird can move up or down
 - ▶ It is controlled by a PS/2 mouse.
 - ▶ If the bird is not flapping, it will free-fall towards the ground.
 - ▶ The bird must not touch anything when flying, otherwise, it will lose life points.
- The game may consist of different types of obstacles and gifts
 - ▶ pipes
 - ▶ dollars, medicine boxes, special flying abilities
- The screen must be kept in motion from the right-hand side to the left-hand side.
 - ▶ The speed increases with the game level
- The level of difficulty can be controlled by other criteria
 - ▶ The types of obstacles

Mini Project Objective

Game Modes

- **Training Mode**

- ▶ Allows the player to practice at the lowest game level.
- ▶ Will continue for a specific time.

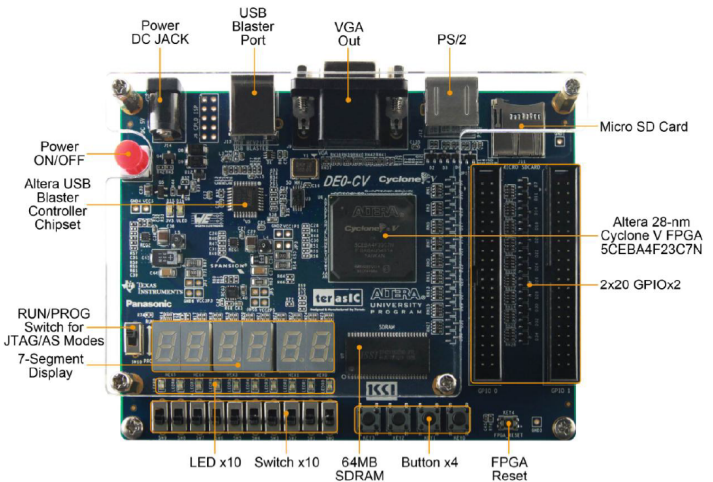
- **Single-player Game Mode**

- ▶ The game will proceed to more advanced levels following certain criteria.
- ▶ The time, distance, or the number of obstacles passed could be used as such criteria.

The game mode can be determined by using a DIP switch on the console or through a selection on the welcome screen.

DE0-CV Board

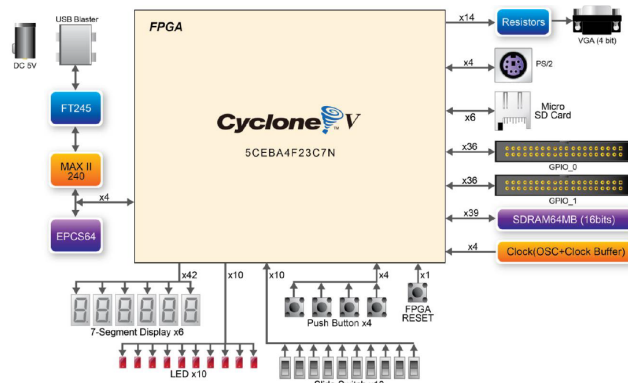
The hardware platform that you will use for implementing the game console is Terasic DE0-CV board.



DE0-CV Board

DE0-CV board includes Altera **Cyclone V 5CEBA4F23C7N** FPGA device

- All the connections are made through the Cyclone V FPGA device.
 - ▶ Gives the flexibility to the user to configure the FPGA to implement any system design.
- The DE0-CV board includes **50 MHz** clock signal.



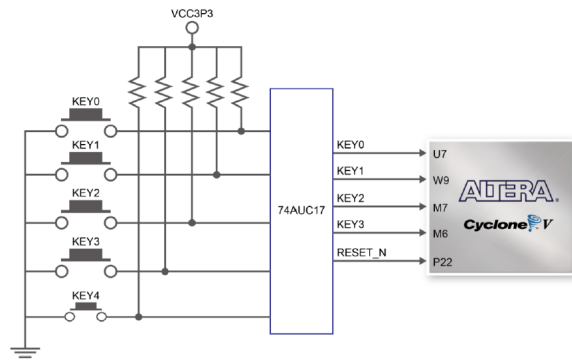
DE0-CV Board - Switches

- There are 10 slide switches (sliders) on the DE0-CV board.
- These switches are used as level-sensitive data inputs to a circuit.
 - ▶ When a switch is in the **DOWN** position it provides a **low logic level**.
 - ▶ When the switch is in the **UP** position it provides a **high logic level**.
- The FPGA pins connected to these switches (and any other I/O) should be defined in the project through pin assignment.



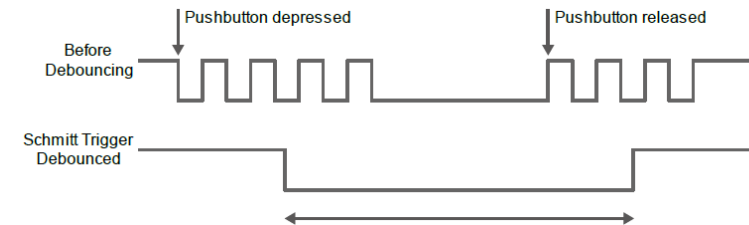
DE0-CV Board - Push-buttons

- The DE0-CV board provides four user-defined push-buttons and one FPGA reset button.
- KEY0, KEY1, KEY2, KEY3, and RESET_N are directly connected to the Cyclone V FPGA as an input.
 - Each button provides a **high** logic level when it is **not pressed**.
 - The button provides a **low** logic level when it is **pressed**.



DE0-CV Board - Push-buttons

- KEY0, KEY1, KEY2, KEY3, and RESET_N are debounced using a Schmitt Trigger circuit.



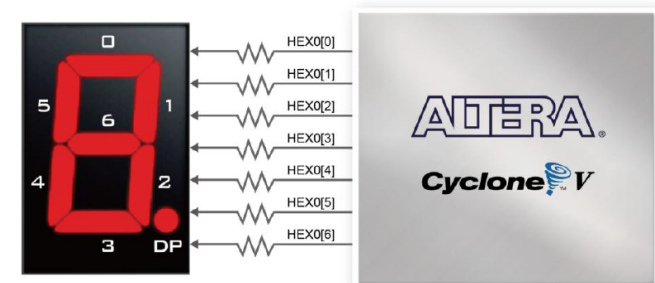
DE0-CV Board - LEDs

- There are 10 user-controllable LEDs on the DE0-CV board.
- Each LED is driven directly by a pin on the Cyclone V FPGA.
 - Driving associated pin to a **high** logic level turns the LED **on**.
 - Driving the pin **low** turns it **off**.



DE0-CV Board - Seven Segments

- The DE0-CV board has six 7-segment displays.
- They are connected to pins on the Cyclone V FPGA.
 - Applying a **low** logic level to a segment causes it to **light up**.
 - Applying a **high** logic level turns it **off**.
- Each segment in a display is identified by an index from 0 to 6.



DE0-CV Board - FPGA Configuration

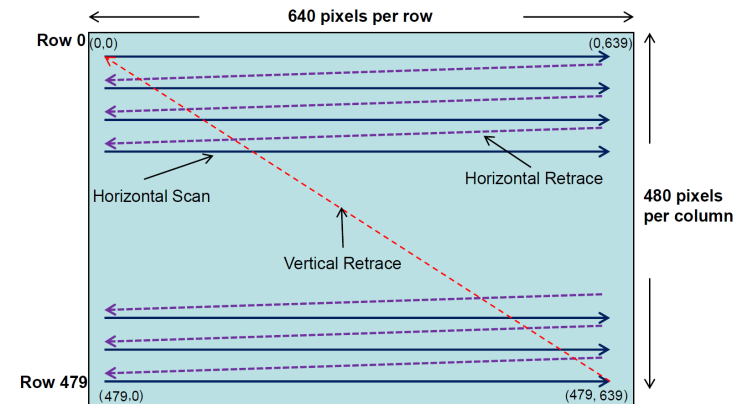
There are two different modes for configuring the **Cyclone V FPGA** on DE0-CV board.

- **JTAG programming**
 - ▶ The configuration bit stream is downloaded **directly** into the **Cyclone V FPGA**.
 - ▶ The FPGA will retain this configuration as long as power is applied to the board.
 - ▶ The configuration is lost when the power is turned off.
- **Active Serial programming**
 - ▶ The configuration bit stream is downloaded into the Altera **EPCS64 serial EEPROM chip**.
 - ▶ It provides non-volatile storage of the bit stream.
 - ▶ When the board is turned on, the configuration data in the EPCS64 device is automatically loaded into the Cyclone V FPGA.

Configuration bit stream is downloaded into the board through the USB Blaster.

VGA Interface

- **VGA (Video Graphics Array)** is a popular display standard developed by IBM and introduced in 1987.
- The resolution of the VGA screen can vary but a standard default size is **640x480 pixels**.
- The screen refreshes the display from left to right, top to bottom.



VGA Interface

Image on VGA screen is displayed by turning the pixels ON and OFF.

- Video signal must redraw the entire screen 60 times per sec (**60Hz**) to avoid flickers.
 - ▶ Human eyes detect flickers at refresh rate less than 30Hz.
- We will use the common VGA display standard at **25MHz** pixel rate with **640x480** resolution.
 - ▶ Each pixel takes 40ns at 25MHz pixel rate.

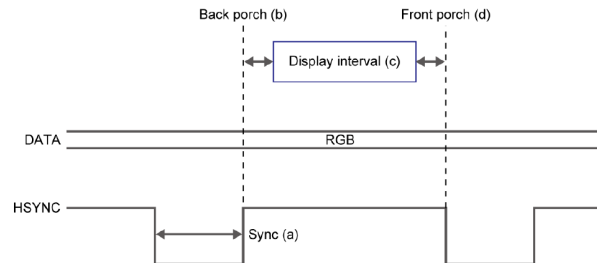
VGA Interface

VGA video standard contains 5 active signals:

- **Horizontal** and **vertical synchronisation** signals.
- Three analog signals for **red**, **green** and **blue (RGB)** colours formation.
 - ▶ By changing the analog voltage levels of the RGB signals, different colours can be produced.
 - ▶ Depending on the number of bits supported by the development board, different amount of colours can be represented.

VGA Interface - Horizontal Synchronisation

- **Horizontal sync** signifies the end of one row of data (i.e. 640 pixels) and the start of the next.
 - ▶ The data (RGB) inputs on the monitor must be off for a time period called the **back porch** (b) after the hsync pulse occurs.
 - ▶ During the data **display interval** (c) the **RGB** data drives each pixel in turn across the row being displayed.
 - ★ When data display interval is finished, the beam returns from the right most position to left most. During the return process, no pixel data is displayed.
 - ▶ **Front porch** (d) is a time period where the RGB signals must again be off before the next hsync pulse can occur.



VGA Interface - Horizontal Synchronisation

- **Horizontal sync** (a) corresponds to 96 pixels.
- **Back porch** (b) corresponds to 48 pixels.
- **Display interval** (c) corresponds to 640 pixels.
- **Front porch** (d) corresponds to 16 pixels.

VGA Horizontal Timing Specification

VGA mode		Horizontal Timing Spec				
Configuration	Resolution(HxV)	a(pixel clock cycle)	b(pixel clock cycle)	c(pixel clock cycle)	d(pixel clock cycle)	Pixel clock(MHz)
VGA(60Hz)	640x480	96	48	640	16	25

VGA Interface - Vertical Synchronisation

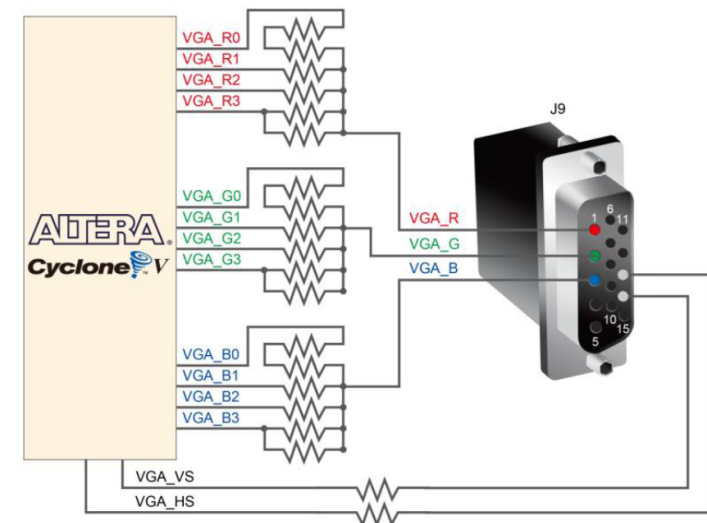
Vertical sync pulse signifies the end of one frame and the start of the next, and the data refers to the set of rows in the frame.

- **Vertical sync** (a) corresponds to 2 lines.
- **Back porch** (b) corresponds to 33 lines.
- **Display interval** (c) corresponds to 480 lines.
- **Front porch** (d) corresponds to 10 lines.

VGA Vertical Timing Specification

VGA mode		Vertical Timing Spec				
Configuration	Resolution(HxV)	a(lines)	b(lines)	c(lines)	d(lines)	Pixel clock(MHz)
VGA(60Hz)	640x480	2	33	480	10	25

VGA Interface - DE0-CV Board



VGA Interface - VGA_Sync Component

We need a component to drive the control signals to the display and provide pixel values at the right rate.

- In order to generate the VGA signal at 25 MHz, the clock signal provided by DE0-CV (50MHz) needs to be halved.
- 25 MHz clock signal can be used by counters to generate the horizontal and vertical sync signals.
- The counters also represent row and column address of a pixel, which can be used by other components to retrieve pixel information.

VGA Interface - VGA_Sync Component

```

6
7 ENTITY VGA_SYNC IS
8   PORT(
9       clock_25Mhz: IN STD_LOGIC;
10      red_out, green_out, blue_out, horiz_sync_out, vert_sync_out: OUT STD_LOGIC;
11      pixel_row, pixel_column: OUT STD_LOGIC_VECTOR(9 DOWNTO 0));
12 END VGA_SYNC;
13
14 ARCHITECTURE a OF VGA_SYNC IS
15   SIGNAL horiz_sync: STD_LOGIC;
16   SIGNAL video_on, video_on_v, video_on_h: STD_LOGIC;
17   SIGNAL h_count, v_count: STD_LOGIC_VECTOR(9 DOWNTO 0);
18
19 BEGIN
20   -- video_on is high only when RGB data is displayed
21   video_on <= video_on_h AND video_on_v;
22
23
24
25
26 PROCESS
27 BEGIN
28   WAIT UNTIL (clock_25Mhz'EVENT) AND (clock_25Mhz='1');
29
30   --Generate Horizontal and Vertical Timing Signals for Video Signal
31   -- H_count counts pixels (640 + extra time for sync signals)
32   --
33   Horiz_sync <= '0';
34   H_count <= 0;
35   IF (h_count = 799) THEN
36     h_count <= "0000000000";
37   ELSE
38     h_count <= h_count + 1;
39   END IF;
40
41   --Generate Horizontal Sync Signal using H_count
42   IF (h_count <= 755) AND (h_count >= 659) THEN
43     horiz_sync <= '0';
44   ELSE
45     horiz_sync <= '1';
46   END IF;

```

VGA Interface - VGA_Sync Component

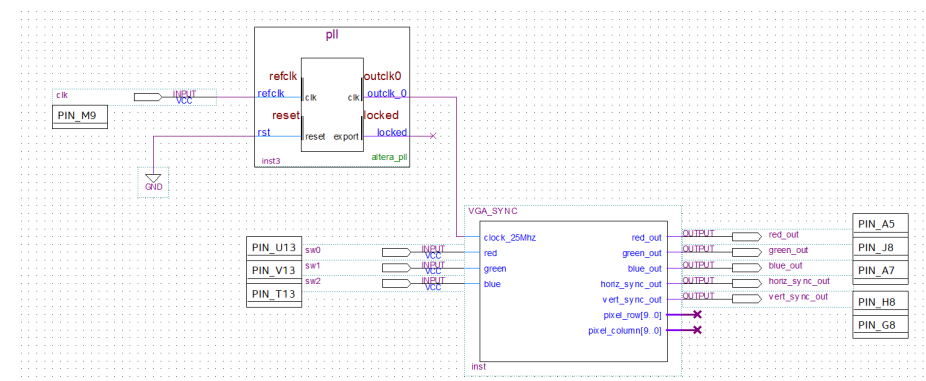
```

48 --V_count counts rows of pixels (480 + extra time for sync signals)
49 --
50 -- Vert_sync
51 -- V_count
52 --
53 IF (v_count >= 524) AND (h_count >= 659) THEN
54   v_count <= "0000000000";
55 ELSEIF (h_count = 659) THEN
56   v_count <= v_count + 1;
57 END IF;
58
59 -- Generate Vertical Sync Signal using V_count
60 IF (v_count <= 494) AND (v_count >= 493) THEN
61   vert_sync <= '0';
62 ELSE
63   vert_sync <= '1';
64 END IF;
65
66 -- Generate Video on Screen Signals for Pixel Data
67 IF (h_count <= 639) THEN
68   video_on_h <= '1';
69   pixel_column <= h_count;
70 ELSE
71   video_on_h <= '0';
72 END IF;
73
74 IF (v_count <= 479) THEN
75   video_on_v <= '1';
76   pixel_row <= v_count;
77 ELSE
78   video_on_v <= '0';
79 END IF;
80
81 -- Put all video signals through DFFs to eliminate any delays that cause a blurry image
82 red_out <= red AND video_on;
83 green_out <= green AND video_on;
84 blue_out <= blue AND video_on;
85 horiz_sync_out <= horiz_sync;
86 vert_sync_out <= vert_sync;
87
88 END PROCESS;
89 END a;
90

```

VGA Interface - Example

Try this simple example and see how you can change the background colour on your VGA screen by using three switches on the DE0-CV board:



Graphics Display on VGA Screen

- Red, Green, and Blue values for all the pixels on the screen should be generated.
- Each pixel is identified by its row and column values.
- These values are generated by VGA_Sync component.
- Horizontal and vertical sync signals are generated by VGA_Sync component.
 - 25MHz clock signal is required.

That means we can draw any object on the screen if we know the RGB values and pixel information of the object.

Draw a 4x4 blue square on the top right of the screen

For pixels within the $0 \leq \text{row} \leq 3$ and $636 \leq \text{column} \leq 639$, the **Blue** signal should be driven to '1'.

Graphics Display - Ball Example

- (x,y) position of the square are set to some constant values.
- Background colour and ball colour are defined as white and red respectively.

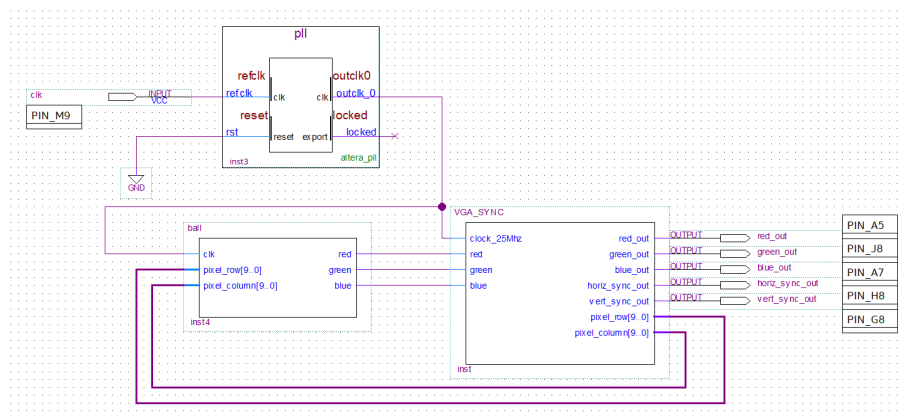
```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.all;
3  USE IEEE.STD_LOGIC_ARITH.all;
4  USE IEEE.STD_LOGIC_UNSIGNED.all;
5
6  ENTITY ball IS
7
8  PORT
9      ( clk           : IN std_logic;
10        pixel_row, pixel_column : IN std_logic_vector(9 DOWNTO 0);
11        red, green, blue       : OUT std_logic);
12  END ball;
13
14  ARCHITECTURE behavior OF ball IS
15
16  SIGNAL ball_on           : std_logic;
17  SIGNAL size              : std_logic_vector(9 DOWNTO 0);
18  SIGNAL ball_y_pos, ball_x_pos : std_logic_vector(9 DOWNTO 0);
19
20  BEGIN
21
22  size <= CONV_STD_LOGIC_VECTOR(4,10);
23  -- ball_x_pos and ball_y_pos show the (x,y) for the centre of ball
24  ball_x_pos <= CONV_STD_LOGIC_VECTOR(550,10);
25  ball_y_pos <= CONV_STD_LOGIC_VECTOR(350,10);
26
27  ball_on <= '1' WHEN ((('0' & ball_x_pos <= pixel_column + size) AND ('0' & pixel_column <= ball_x_pos + size)
28  AND ('0' & ball_y_pos <= pixel_row + size) AND ('0' & pixel_row <= ball_y_pos + size)) ELSE
29  '0';
30
31  -- Colours for pixel data on video signal
32  -- Keeping background white and square in red
33  Red <= '1';
34  -- Turn off Green and Blue when displaying square
35  Green <= NOT ball_on;
36  Blue <= NOT ball_on;
37
38  END behavior;

```

Graphics Display - Ball Example

Try this example to see the red square on white background. You may change the colour and position of the square in ball component.



Graphics Display - Bouncy Ball Example

The motion feature is added to our simple object to make it bounce off the edges.

- The new position of the ball should be updated once for each frame.
 - One update per each vertical sync.
- Ball position is calculated by adding its current Y position and its vertical motion.
- Screen boundaries are checked; ball speed is changed once it reaches the boundaries at row 0 and 479.
- Two pushbuttons are used to change the background and ball colour.

Graphics Display - Bouncy Ball Example

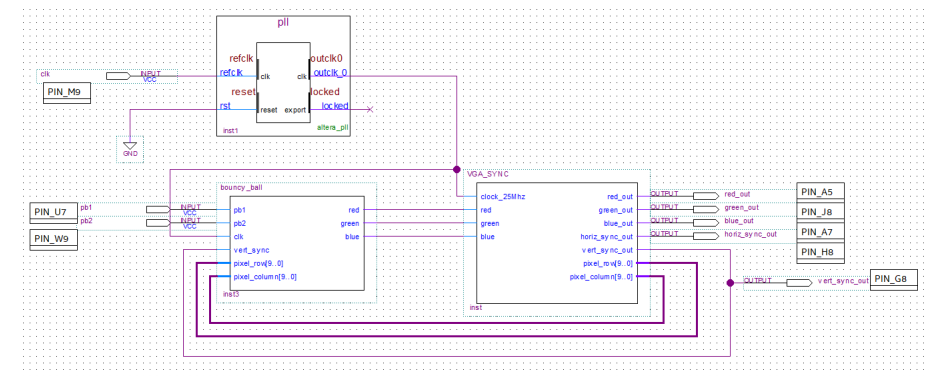
```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.all;
3  USE IEEE.STD_LOGIC_ARITH.all;
4  USE IEEE.STD_LOGIC_SIGNED.all;
5
6
7  ENTITY bouncy_ball IS
8  PORT
9      ( pb1, pb2, clk, vert_sync : IN std_logic;
10        pixel_row, pixel_column : IN std_logic_vector(9 DOWNTO 0);
11        red, green, blue        : OUT std_logic);
12  END bouncy_ball;
13
14  architecture behavior of bouncy_ball is
15
16  SIGNAL ball_on : std_logic;
17  SIGNAL size : std_logic_vector(9 DOWNTO 0);
18  SIGNAL ball_v_pos : std_logic_vector(9 DOWNTO 0);
19  SIGNAL ball_x_pos : std_logic_vector(10 DOWNTO 0);
20  SIGNAL ball_y_motion : std_logic_vector(9 DOWNTO 0);
21
22  BEGIN
23
24      size <= CONV_STD_LOGIC_VECTOR(8,10);
25      -- ball_x_pos and ball_y_pos show the (x,y) for the centre of ball
26      ball_x_pos <= CONV_STD_LOGIC_VECTOR(50,11);
27
28      ball_on <= '1' when ( ('0' & ball_x_pos <= '0' & pixel_column + size) and ('0' & pixel_column <= '0' & ball_x_pos + size)
29                        and ('0' & ball_y_pos <= pixel_row + size) and ('0' & pixel_row <= ball_y_pos + size) ) else
30                        '0';
31
32      -- Colours for pixel data on video signal
33      -- Changing the background and ball colour by pushbuttons
34      Red <= pb1;
35      Green <= (not pb2) and (not ball_on);
36      Blue <= not ball_on;
37
38
39      Move_Ball: process (vert_sync)
40      begin
41          -- Move ball once every vertical sync
42          if (rising_edge(vert_sync)) then
43              -- Bounce off top or bottom of the screen
44              if ( ('0' & ball_y_pos >= CONV_STD_LOGIC_VECTOR(479,10) - size) ) then
45                  ball_y_motion <= - CONV_STD_LOGIC_VECTOR(2,10);
46              elsif (ball_y_pos <= size) then
47                  ball_y_motion <= CONV_STD_LOGIC_VECTOR(2,10);
48              end if;
49              -- Compute next ball Y position
50              ball_y_pos <= ball_y_pos + ball_y_motion;
51          end if;
52      end process Move_Ball;
53
54  END behavior;
55

```

Graphics Display - Bouncy Ball Example

Try this example and see how you can change the colour of background and bouncy ball by using **KEY 0** and **KEY 1** on DE0-CV board:



Graphics Display - Bouncy Ball Example

- When **no button** is pressed:
 - Pixels showing the ball has R='1', G='0', B='0': **Red ball**
 - Background pixels has R='1', G='0', B='1': **Magenta background**
- When only **pushbutton 1** is pressed:
 - Pixels showing the ball has R='0', G='0', B='0': **Black ball**
 - Background pixels has R='0', G='0', B='1': **Blue background**
- When only **pushbutton 2** is pressed:
 - Pixels showing the ball has R='1', G='0', B='0': **Red ball**
 - Background pixels has R='1', G='1', B='1': White background
- When both **pushbutton 1 and 2** are pressed:
 - Pixels showing the ball has R='0', G='0', B='0': **Black ball**
 - Background pixels has R='0', G='1', B='1': **Cyan background**

Summary

- We discussed the mini project and its objective.
- We talked about DE0-CV board and its interfaces.
- We looked at VGA interface and discussed how to show graphics on the VGA screen through several examples.

Acknowledgment

- Some figures/notes are taken from or inspired by the
 - ▶ CS305 Lecture notes by Muhammad Nadeem, 2019